Felex v1.0.1 technical documentation

Abstract

Felex v1.0.1 is a local software system for feed ration calculation, analysis, and optimization for farm animals. The system combines a curated feed library, a nutrient calculation module, an economic analysis module, a multi-stage ration optimizer, price control tools, document export, and an AI assistant connected to a local Ollama server. This document describes the system architecture, data model, computational procedures, validation workflow, and current limitations in a form suitable for a technical appendix or scientific paper.

1. System purpose

The system is intended to support the following tasks:

- storage and inspection of feed profiles;
- ration formulation for different animal groups and production types;
- calculation of energy, protein, fiber, minerals, and vitamins supplied by the ration;
- comparison of actual composition against reference norms;
- ration cost assessment;
- ration optimization by cost, priority nutrient balance, or fixed-feed rebalance;
- documentation of results in PDF, CSV, and XLSX formats.

2. Architecture

2.1. General structure

Felex consists of four major subsystems:

Subsystem	Role
Frontend	user interface implemented with React/Vite
Embedded API	local HTTP API implemented in Rust/Axum
Desktop shell	Tauri wrapper for export and external-link commands
Local AI	local LLM backend through Ollama

The frontend is the main working environment for the operator. In desktop mode it communicates both with the embedded HTTP API and with Tauri commands.

2.2. Technology stack

- **Frontend**: React, TypeScript, Zustand, i18next, React Markdown.
- **Backend**: Rust, Axum, rusqlite.
- **Desktop layer**: Tauri 2.
- **Export**: `printpdf`, `rust_xlsxwriter`, CSV generation.
- **LLM**: Ollama API (`/api/chat`).

3. Data model

3.1. `Feed` entity

The feed object stores:

- identifiers and source metadata;
- Russian and English names;
- category and subcategory;
- nutrient indicators;
- economic indicators;
- verification and customization flags.

The main nutrient groups include:

- dry matter;
- metabolizable energy;
- crude and digestible protein;
- fiber;
- calcium, phosphorus, and other macroelements;
- trace elements;
- vitamins.

3.2. `FeedPrice` entity

Current price data are stored separately from the base feed profile. A price record contains:

- `feed_id`;
- region;
- price per ton;
- price date;
- source type;
- free-form notes;
- history records for auditing.

This separation allows price updates without modifying the underlying nutrient profile.

3.3. Ration entity

A ration stores:

- the animal-group identifier;
- animal properties;
- head count;
- the list of feeds;
- amount of each feed per head per day;
- lock flags for selected ingredients;
- the active norm preset;
- user adjustments to norms.

4. Normative basis

The system uses built-in presets for the main production groups. For each indicator the reference may contain:

- minimum;
- target;
- maximum.

The calculation module compares the actual value against that interval and produces a status label. The user may:

- use the built-in preset;
- select another preset as the balancing reference;
- manually override individual bounds and targets.

5. Nutrient calculation

5.1. Core principle

For each ingredient, the contribution of each nutrient is computed as the product of feed amount and nutrient concentration. The ration total is the sum across all ingredients.

For nutrient `j` in a ration with `n` ingredients:

`N_j = Σ(i=1..n) q_i * c_ij`

where:

- `q_i` is the amount of feed `i` per head per day;
- `c_ij` is the concentration of nutrient `j` in feed `i`;
- `N_j` is the total intake of nutrient `j`.

5.2. Group scaling

If head count equals `H`, total group feed use and cost are computed as:

- `Q_group = H * Q_head`
- `C_group = H * C_head`

This scaling is applied consistently in the UI, the status bar, and exported reports.

6. Economic module

The economic module uses price per ton. Price per kilogram is obtained by division by 1000. Daily ingredient cost per head is calculated as:

`Cost_i = q_i * (Price_i / 1000)`

Total ration cost per head:

`Cost_head = Σ(i=1..n) Cost_i`

Total ration cost for the group:

`Cost_group = H * Cost_head`

The system also derives secondary report indicators:

- total ration mass per head;
- total ration mass per group;
- daily cost per head;
- daily cost per group;
- monthly and annual group estimates.

7. Price subsystem

7.1. Price layers

The price subsystem is multi-layered:

1. a curated seed dataset shipped with the application;
2. an automatic web price fetcher;
3. manual user input.

7.2. Normalization and mapping

The web fetcher normalizes feed names through case folding, whitespace cleanup, and Russian orthographic normalization (`e/ё`). Prices are then matched to feed records by canonical keys and by approximate name correspondence.

7.3. Source-link relevance control

Version 1.0 strengthens source validation:

- web-search results are filtered by feed-specific keywords;
- direct links are shown only for feed-specific sources;
- benchmark-based prices do not expose misleading direct links;
- instead, the operator receives a feed-specific validation search.

This design reduces the risk of interpreting a benchmark price as if it came from an exact product page.

8. Optimization module

8.1. Operating modes

The backend exposes three optimization modes:

- `minimize_cost` solves a classic linear program over the current unlocked feed set;
- `balance` runs the hybrid multi-stage nutrient-balancing workflow and is the default interactive mode;
- `fixed` uses the same balancing logic with tighter feed-movement limits so the ration stays

close to the operator draft.

8.2. Stage 0: starter ration for empty drafts

If the ration is empty and the user starts optimization from the interface, the backend first builds a starter plan from the feed library.

- feeds are classified into roughage, succulent, concentrate, protein, animal-origin, mineral, premix, vitamin, and other groups;
- the planner selects group-appropriate feeds for the active animal group and derives starting amounts from a template structure;
- the optimization response marks such runs with `auto_populated = true` and appends `+starter_plan` to `applied_strategy`.

8.3. Stage 1: screening and feasibility check

Before final balancing, the current feed set is screened against the active norm set.

- the screening pass identifies limiting nutrients that cannot be covered by the current ingredients;
- candidate additions are ranked from the feed library and returned as `FeedRecommendation` records with reason, category, priority, and suggested amount;
- truly infeasible cases are no longer masked as unchanged "feasible" rations; the solver returns `Infeasible`.

8.4. Stage 2: priority-tiered balancing

The balancing solver no longer penalizes amount changes as a primary objective. Instead it uses bounded feed movement and species-specific lexicographic passes.

- cattle: Tier 1 `energy_eke`, `crude_protein`, `dig_protein_cattle`; Tier 2 `ndf_pct_dm`, `calcium`, `phosphorus`; Tier 3 `selenium`, `vit_a`, `vit_d3`, `vit_e`;
- swine: Tier 1 `energy_oe_pig`, `lysine_sid`, `crude_protein_pct`; Tier 2 `methionine_cystine_sid`, `calcium`, `phosphorus_avail`; Tier 3 `vit_e`;
- poultry: Tier 1 `energy_oe_poultry`, `crude_protein_pct`, `lysine_tid_pct`, `methionine_cystine_tid_pct`; Tier 2 `calcium_pct`, `phosphorus_avail_pct`; Tier 3 `vit_e`.

Balance mode allows each unlocked feed to move within `max(±60%, ±5 kg)` of the current amount. Fixed mode tightens this envelope to `max(±25%, ±2.5 kg)`.

8.5. Stage 3: economic pass and result payload

After the nutrient tiers are locked within tolerance bands, a final cost pass searches the feasible region for the cheapest acceptable solution.

The returned `DietSolution` contains:

- updated ingredient amounts and nutrient summary;
- daily cost and validation warnings;

- `recommendations` from feed screening;
- `applied_strategy` describing the executed workflow;
- `auto_populated` to indicate that a starter ration was injected before optimization.

8.6. Result interpretation

- `Optimal`: the staged solver found a ration inside the active bounds and nutrient bands.
- `Feasible`: used for the unchanged current ration when no optimization step is applicable.
- `Infeasible`: the current ingredient set cannot satisfy the active norms inside the allowed movement bounds. The intended next step is to add recommended feeds or restart from auto-populate.
- `Unbounded` and `Error`: reserved for solver or transport failures.

9. Local LLM integration

9.1. Assistant role

The LLM does not perform the core nutrient math. Its function is to:

- provide textual interpretation of ration balance;
- explain deficits and excesses;
- suggest feed substitutions;
- answer domain-specific user questions.

9.2. Built-in reasoning mode

The built-in agent keeps model reasoning mode enabled. Ollama chat requests are sent with `think: true`, and the backend accepts separate reasoning and final-answer fields instead of disabling thinking to avoid transport failures.

- the agent waits for final user-facing content even when the model also emits a separate reasoning stream;
- reasoning-only completions that stop because of context length are retried instead of being surfaced as empty answers;
- the frontend now applies user context-size changes to the backend request path, so `num_ctx` updates are effective immediately.

9.3. Robustness against Ollama failures

Large local models such as Qwen 3.5 9B may still trigger `500 Internal Server Error` or runner-resource failures in Ollama. The backend therefore applies a retry ladder.

1. send the request with the configured `num_ctx`;
2. on 5xx responses, retry with smaller context sizes down to `1024`;
3. if Ollama terminates the `qwen3.5:9b` runner because of local resource pressure, retry the same request on `qwen3.5:4b`;
4. if all retries fail, return a diagnostic error instead of a blank answer.

This keeps the agent responsive without switching off reasoning mode.

10. Export subsystem

10.1. Supported formats

The system exports:

- PDF;
- CSV;
- XLSX.

10.2. Export parameters

The user may control:

- destination directory;
- file name;
- font family;
- visual appearance profile.

10.3. Report payload

An exported report may contain:

- title and metadata;
- ration composition;
- per-head indicators;
- group indicators;
- norm comparison;
- economics;
- free-form notes.

11. Interface and bilingual support

The system supports both Russian and English through:

- localized UI dictionaries;
- bilingual documentation metadata;
- bilingual Markdown and PDF copies of the embedded help documents.

This is important both for local operational use and for English-language reporting, appendices, and publications.

12. Reliability, reproducibility, and benchmark methodology

Several design choices support reproducibility:

- fixed data structures for feeds and norms;
- price history persistence;
- explicit separation between manual and automatically acquired prices;

- export to documented file formats;
- embedded technical documentation aligned with interface terminology;
- benchmark snapshots stored alongside the source tree, including [`docs/benchmarks/2026-03-11-perf_probe.json`](../../docs/benchmarks/2026-03-11-perf_probe.json).

12.1. Benchmark method used on March 11, 2026

Benchmarks were executed on March 11, 2026 with `cargo run --quiet --bin perf_probe` against the local feed database (300 feeds).

- nutrient-calculation microbenchmarks used `25,000`, `15,000`, and `8,000` iterations for 8-, 24-, and 64-ingredient synthetic rations;
- optimization benchmarks used `300` iterations for `minimize_cost` and `200` iterations for `balance` and `fixed` across three real scenarios: `dairy_low_fiber`, `grower_pig_lysine_deficit`, and `layer_low_calcium`;
- workflow helpers used `500` iterations for starter-plan generation and `500` iterations for feed screening;
- agent benchmarks called `/api/v1/agent/chat` once per scenario for `qwen3.5:4b` (`num_ctx = 8192`) and `qwen3.5:9b` (`num_ctx = 4096`).

12.2. Latest benchmark results

Nutrient calculation:

Scenario	Mean time	Throughput
8 ingredients	`0.000385 ms`	`2.60 million calls/s`
24 ingredients	`0.001108 ms`	`0.90 million calls/s`
64 ingredients	`0.002912 ms`	`0.34 million calls/s`

Optimizer:

Scenario	Mode	Mean time	Final status
`dairy_low_fiber`	`minimize_cost`	`0.214929 ms`	`Infeasible`
`dairy_low_fiber`	`balance`	`0.978980 ms`	`Infeasible`
`dairy_low_fiber`	`fixed`	`1.016516 ms`	`Infeasible`
`grower_pig_lysine_deficit`	`minimize_cost`	`0.177419 ms`	`Infeasible`
`grower_pig_lysine_deficit`	`balance`	`0.688114 ms`	`Infeasible`
`grower_pig_lysine_deficit`	`fixed`	`0.715599 ms`	`Infeasible`
`layer_low_calcium`	`minimize_cost`	`0.075507 ms`	`Optimal`
`layer_low_calcium`	`balance`	`0.477134 ms`	`Optimal`
`layer_low_calcium`	`fixed`	`0.469067 ms`	`Optimal`

Workflow helpers:

Operation	Scenario	Mean time
`auto_populate_preview`	`starter_dairy`	`50.688338 ms`

`auto_populate_preview`	`starter_swine`	`31.849788 ms`
`auto_populate_preview`	`starter_layer`	`32.868358 ms`
`screen`	`dairy_low_fiber`	`24.946853 ms`
`screen`	`grower_pig_lysine_deficit`	`24.694417 ms`
`screen`	`layer_low_calcium`	`32.404530 ms`

Agent benchmarks:

Model	Scenario	Context	Latency	Outcome
`qwen3.5:4b`	`dairy_low_fiber`	`8192`	`84.23 s`	detailed answer
`qwen3.5:4b`	`grower_pig_lysine_deficit`	`8192`	`16.75 s`	response returned, but only a short preamble
`qwen3.5:4b`	`layer_low_calcium`	`8192`	`54.19 s`	detailed answer
`qwen3.5:9b`	`dairy_low_fiber`	`4096`	`86.26 s`	detailed answer
`qwen3.5:9b`	`grower_pig_lysine_deficit`	`4096`	`78.57 s`	detailed answer
`qwen3.5:9b`	`layer_low_calcium`	`4096`	`74.71 s`	detailed answer

The key consequence of these results is that the redesigned optimizer is computationally inexpensive; the limiting factor in difficult scenarios is feed-set feasibility, not solver runtime.

13. Current limitations

The current version is limited by:

- incomplete market coverage for some specialized feeds;
- optimization quality still depends on feed-profile completeness;
- two benchmark scenarios remain genuinely infeasible with their fixed feed sets; the system now reports this explicitly instead of hiding it as a no-op solution;
- `max_inclusion_*` feed limits exist in the database model but are not yet enforced by the optimizer;
- `DCadOutOfRange` exists in the validation model, but functional DCAD calculation is not yet implemented;
- poultry presets still require manual verification for high-performance flocks because part of the normative base is older than current commercial guidelines;
- the inability of a purely computational approach to replace expert biological interpretation;
- local LLM latency and answer quality vary strongly by model size and hardware; the 4B model is faster but may under-answer analytical prompts.

14. Future directions

Priority development directions include:

- broader regional price analytics;
- seasonal price modeling;
- enforcement of `max_inclusion_*` and other practical inclusion constraints inside the optimizer;

- implementation of DCAD and other transition-cow checks;
- updates to dairy and poultry normative presets where newer references are available;
- automated prompt-quality evaluation for the built-in agent in addition to transport-level benchmarks;
- automated generation of standardized reports for farms and experimental studies.

15. Conclusion

Felex v1.0.1 implements an integrated computational pipeline: `feed data -> nutrient calculation -> starter ration -> screening -> tiered optimization -> cost pass -> validation -> documentation -> local AI interpretation`. The redesigned optimizer fixes the previous "feeds do not move" behavior, exposes infeasible feed sets instead of masking them as unchanged solutions, and pairs the backend solver with a built-in agent that keeps reasoning mode enabled while recovering from local Ollama failures. The system remains suitable both for practical ration work and for technical or scientific reporting on digital ration balancing.